

Card Match Game – Architecture Guide

Development Time: 14-15hrs approx

This document summarizes the code structure, responsibilities, core flows, and safe enhancements.
Structure (Assets/Scripts)

- **Manager**

- GameManager.cs: Orchestrates start/restart/save/load/game over; holds rows, cols, cardManager, saveManager, eventDispatcher. Emits events (GAME_STARTED, GAME_OVER, GAME_SAVED) via EventDispatcher. DefaultExecutionOrder may be used to initialize early.
- CardManager.cs: Builds grid, initializes cards from sprites, registers with MatchController, triggers initial reveal. Applies grid cell size from GameLayoutScriptableObject. Uses GameManager.Instance assets.
- MatchController.cs: Handles clicks, selection, match/mismatch, input lock, initial reveal, game-over check. Dispatches GAME_CARD_MATCH / GAME_CARD_MISMATCH.
- ScoreManager.cs: Tracks Turns, Matches, Combo, Score. On match: Turns++, Matches++, Combo++, Score += (Combo > 1 ? 1*Combo : 1); on mismatch: Turns++, Combo = 0. Dispatches GAME_SCORE_UPDATE with IScoreData.
- SaveLoadManager (+ ISaveManager): JSON persistence used by GameManager (Save/Load/Delete/HasSave).

- **UI**

- UIScreenBase.cs: Base for screens; show/hide.
- MainMenuUIHandler.cs: Populates layouts from GameLayoutScriptableObject; starts or resumes a game. Confirms load when a save exists.
- GameplayUIHandler.cs: Subscribes to GAME_STARTED to spawn grid; updates HUD on GAME_SCORE_UPDATE.
- GameEndUIHandler.cs: Final stats; replay/home; releases pooled cards.
- CardUI.cs: Card visuals; click gating; flip animation using X-scale squash (1→0→1) with midpoint swap; matched alpha set to 0f.

- **Utils**

- EventSystem/EventDispatcher.cs: Scene event bus (Subscribe/Dispatch/UnsubscribeAll). MonoBehaviour for lifecycle and Inspector wiring.
- PoolSystem/ObjectPool.cs (+ IObjectPool): Pool for card objects (initialize, get, release all).
- SoGeneration/AnimalSpritesScriptable.cs: Front sprites and back sprite.
- SoGeneration/GameLayoutScriptableObject.cs: Allowed rows/cols with per-layout cellSize.
- OptimisationUtils/GameOptimizer.cs: Optional performance controls and monitoring (2D notes below).

Core Flows

- **Start:** MainMenuUIHandler → GameManager.StartGame → Dispatch GAME_STARTED → GameplayUIHandler.SpawnGrid → CardManager.CreateCardGrid → MatchController.Initialise + RunInitialReveal.
- **Matching:** MatchController compares CardUI.CardValue; on match mark both, dispatch GAME_CARD_MATCH; on mismatch dispatch GAME_CARD_MISSMATCH, flip back after delay.
- **Scoring:** ScoreManager listens to card events; updates stats as above; dispatches GAME_SCORE_UPDATE with IScoreData for HUD.
- **HUD:** GameplayUIHandler listens to GAME_STARTED and GAME_SCORE_UPDATE; updates labels (turns, matches, score, combo).
- **Save/Load:** GameManager.GameSave() collects layout/score/card states; GameManager.LoadGame(key) restores grid and score state.

Grid, Sprites, and Matching

- Card pairs are built from sprite indices in AnimalSpritesScriptable. The card's matching key is its value, which equals the SO index used to fetch the sprite.
- Duplicates of the same SO index are allowed; any tiles with the same index will match.
- GridLayoutScriptableObject includes cellSize for each (rows, cols); CardManager applies it to GridLayoutGroup.cellSize for consistent tile sizing.

Events

- GAME_STARTED: payload GameStartedPayload { rows, cols }
- GAME_OVER: payload IScoreData
- GAME_SAVED: payload GameSavedPayload { key }
- GAME_CARD_MATCH / GAME_CARD_MISSMATCH: no payload
- GAME_SCORE_UPDATE: payload IScoreData

Defensive Practices

- Null-guard UI/manager refs and assets; initialize EventDispatcher early so OnEnable subscribers can attach.
- Ensure grid has an even number of cells; prewarm ObjectPool before first GetObjectFromPool.
- Validate sprite indices against AnimalSpritesScriptable.Count.

Enhancements

- Normalize event names in EventDispatcher to avoid string mismatches.
- Create a base class and implement it to pass EventDispatcher to subclasses.
- Add UI to enable/disable Music and Sfx.
- Add Particles on gameover.
- Add a "Continue Game" option on the main menu when a save exists.
- Add fades in UIScreenBase for smoother transitions via coroutine or DOTween.
- Add ReturnToPool(GameObject) in ObjectPool for granular returns.
- Gate verbose logs with #if UNITY_EDITOR.

2D Optimization Notes (GameOptimizer)

- Keep targetFrameRate (e.g., 60). Adaptive frame rate can clamp within 30–60 on mobile.
- gcInterval is in seconds; 10–30s is reasonable.
- Disable or avoid 3D-specific settings in pure 2D: occlusion culling, billboards, high shadow quality; use SpriteAtlas for batching.
- Physics2D: autoSimulation true, autoSyncTransforms false are sensible defaults.

Quick Reference

- Start: GameManager.StartGame(rows, cols)
- Save: GameManager.GameSave()
- Load: GameManager.LoadGame(key)
- Subscribe: EventDispatcher.Subscribe(name, handler); UnsubscribeAll(this) in OnDisable.